

Incentive-Compatible Multi-Shard Mining: Game-Theoretic Analysis of DAG-Based Proof-of-Work with Halving Emission

Q-NarwhalKnight Protocol Team

research@quillon.xyz

March 2026

Version 2.0 (revised)

Abstract

We present a rigorous game-theoretic analysis of the Q-NarwhalKnight multi-shard mining protocol, in which miners compete across K parallel consensus shards with VDF-based difficulty adjustment and an adaptive emission controller that maintains constant annual issuance regardless of aggregate throughput. We model multi-shard mining as a K -shard congestion game and prove that the hash-based shard routing mechanism admits a unique symmetric Nash equilibrium in which miners distribute hashrate proportionally across shards (Theorem 4.3). We establish that the 256-year geometric halving emission schedule (21M QUG cap, 4-year halving, 64 eras) with budget-based error correction is incentive-compatible against a specific class of deviations (throughput manipulation, timestamp attacks) under our model’s assumptions (Theorem 3.11). We further analyze three strategic deviations—shard-hopping, hashrate concentration, and selfish mining—showing that under our model each yields weakly or strictly lower expected payoff than honest mining in the DAG setting (Theorems 5.2–5.6). We formalize the MEV resistance properties of DAG-based parallel block production, showing that single-shard atomic MEV extractable by a rational adversary is $\mathcal{O}(1/K)$ compared to linear-chain protocols, while noting that cross-shard MEV strategies may partially offset this dilution (Theorem 6.3). Finally, we analyze fee market dynamics under multi-shard production, showing that the PPLNS reward distribution is Pareto-optimal and strategy-proof within the class of proportional-share mechanisms (Theorem 7.4).

Contents

1	Introduction	3
1.1	Contributions	3
1.2	Paper Organization	4
2	Mining Model	4
2.1	System Architecture	4
2.2	Block Production	4
3	Emission Schedule and Incentive Compatibility	5
3.1	Geometric Halving Schedule	5
3.2	Budget-Based Error Correction	6
3.3	Lyapunov Stability of Emission	6
3.4	Incentive Compatibility	7
4	The Multi-Shard Mining Game	9
4.1	Congestion Game Formulation	9
4.2	Nash Equilibrium	9
4.3	Deterministic Routing Enforcement	11

5	Strategic Deviations	11
5.1	Shard-Hopping	11
5.2	Hashrate Concentration	11
5.3	Selfish Mining in the DAG Setting	12
6	MEV Resistance of DAG-Based Block Production	13
6.1	MEV in Linear vs. DAG Chains	13
6.2	Quantum Commit-Reveal MEV Shield	15
7	Fee Market Dynamics and Pool Rewards	15
7.1	Fee Distribution Mechanism	15
7.2	PPLNS Reward Distribution	16
7.3	Multi-Shard Fee Market	16
8	Comparison with Related Systems	17
9	Related Work	18
10	Conclusion	18
10.1	Limitations	19
10.2	Future Work	19
A	Proof of Nash Equilibrium Uniqueness via Potential Function	20
B	Emission Controller Parameters	21
C	Notation Summary	22

1 Introduction

The design of mining incentives is a foundational challenge in proof-of-work (PoW) blockchain systems. Since Nakamoto’s original Bitcoin protocol [1], a rich literature has emerged analyzing the game-theoretic properties of mining: selfish mining strategies [2], fee market dynamics [4], miner extractable value (MEV) [3], and the incentive compatibility of emission schedules [5]. However, these analyses overwhelmingly focus on *single-chain* architectures where miners compete for a single block slot per round.

The Q-NarwhalKnight protocol introduces a fundamentally different mining architecture: K parallel consensus shards produce blocks simultaneously, miners are routed to shards via deterministic hash-based assignment, and an adaptive emission controller maintains constant annual issuance regardless of the aggregate block-production rate. This architecture raises novel game-theoretic questions:

- Does multi-shard mining admit a Nash equilibrium, and if so, is it unique?
- Is the adaptive emission schedule incentive-compatible under rational miners?
- Are classical attacks (selfish mining, hashrate concentration) profitable in the DAG setting?
- How does parallel block production affect MEV extraction?
- Is the PPLNS pool reward distribution strategy-proof?

1.1 Contributions

We make the following contributions:

1. **Multi-shard mining equilibrium (Theorem 4.3):** We model K -shard mining as a congestion game and prove existence and uniqueness of a symmetric Nash equilibrium where miners distribute hashrate proportionally across shards.
2. **Emission incentive compatibility (Theorem 3.11):** Under our model’s assumptions (single-period, linear cost, unified coalition), we show that the adaptive emission controller with budget-based error correction is incentive-compatible against a defined class of deviations (throughput manipulation, timestamp attacks). We discuss limitations including difficulty-timing attacks and multi-period strategic considerations.
3. **Emission stability (Theorem 3.9):** We prove Lyapunov stability of the cumulative emission trajectory, showing the error-correction feedback loop converges asymptotically to the target schedule.
4. **Shard-hopping resistance (Theorem 5.2):** Shard-hopping yields zero expected gain over honest mining due to the deterministic routing mechanism.
5. **Hashrate concentration resistance (Theorem 5.4):** Concentrating hashrate on a single shard reduces expected payoff compared to proportional distribution.
6. **Selfish mining resistance (Theorem 5.6):** DAG parallelism with BFT finality eliminates the profitability of selfish mining for any adversary controlling $< 50\%$ of hashrate.
7. **MEV resistance (Theorem 6.3):** Parallel K -shard block production reduces extractable MEV by a factor of K compared to sequential chains.
8. **PPLNS optimality (Theorem 7.4):** The PPLNS reward distribution is Pareto-optimal and strategy-proof in the class of proportional-share mechanisms.

1.2 Paper Organization

Section 2 formalizes the mining model. Section 3 analyzes the emission schedule. Section 4 studies the multi-shard mining game. Section 5 analyzes strategic deviations. Section 6 treats MEV resistance. Section 7 analyzes fee markets and pool rewards. Section 8 compares with related systems. Section 9 surveys related work, and Section 10 concludes.

2 Mining Model

2.1 System Architecture

Definition 2.1 (Multi-Shard Mining System). The Q-NarwhalKnight mining system $\mathcal{M} = (\mathcal{N}, \mathcal{K}, \mathcal{E}, \mathcal{R})$ consists of:

- $\mathcal{N} = \{1, \dots, n\}$: a set of n miners.
- $\mathcal{K} = \{s_1, \dots, s_K\}$: a set of K consensus shards (default $K = 4$).
- $\mathcal{E}$: the emission controller specifying per-block rewards.
- $\mathcal{R}$: the shard routing function.

Definition 2.2 (Miner). Each miner $i \in \mathcal{N}$ is characterized by:

- $h_i \in \mathbb{R}_{\geq 0}$: computational hashrate (hashes per second).
- $c_i \in \mathbb{R}_{> 0}$: cost per hash (electricity + hardware amortization).
- $\sigma_i : \mathcal{K} \rightarrow [0, 1]$: a shard allocation strategy, where $\sigma_i(s_k)$ is the fraction of hashrate allocated to shard s_k , with $\sum_{k=1}^K \sigma_i(s_k) = 1$.

Definition 2.3 (Aggregate Hashrate). The total hashrate on shard s_k is $H_k = \sum_{i \in \mathcal{N}} h_i \cdot \sigma_i(s_k)$. The total network hashrate is $H = \sum_i h_i$.

2.2 Block Production

Definition 2.4 (Mining Challenge). For each shard s_k , a mining challenge is defined by:

$$\text{challenge}_k = (\text{block_hash}_k, \text{height}_k, \text{target}_k, \text{vdf_iters}_k, \text{expires_at}_k)$$

where target_k is the difficulty target (a 256-bit value) and vdf_iters_k specifies the VDF anti-grinding iterations. A miner solves the challenge by finding a nonce ν such that:

$$H(\text{challenge}_k || \nu) \leq \text{target}_k.$$

The expected number of hash evaluations is $\mathbb{E}[\text{attempts}] = 2^{256} / \text{target}_k$.

Definition 2.5 (Shard Routing). The deterministic shard routing function assigns mining solutions to shards:

$$\mathcal{R}(\text{tx}) = H(\text{tx}) \bmod K$$

where H is SHA3-256.

Assumption 2.6 (Honest Majority). At least 2/3 of total hashrate is controlled by honest miners who follow the protocol.

Assumption 2.7 (Rational Miners). Each miner i seeks to maximize expected profit: $\pi_i = \mathbb{E}[\text{rewards}_i] - c_i \cdot h_i$.

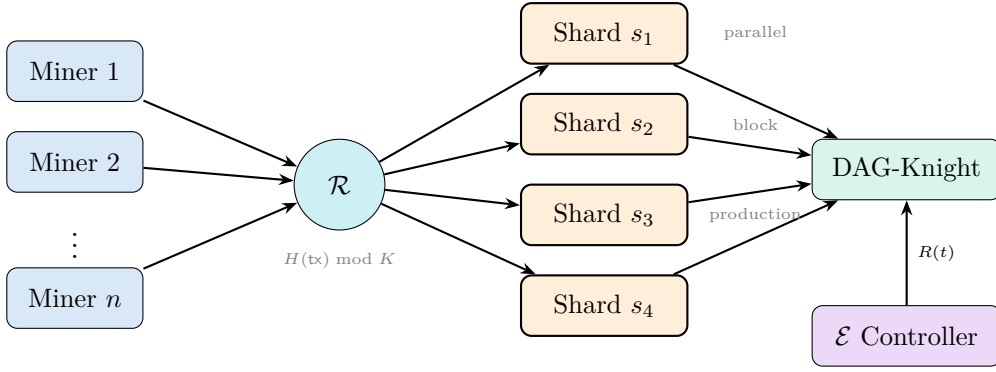


Figure 1: Multi-shard mining architecture: n miners route solutions through K parallel consensus shards into the DAG-Knight consensus layer. The emission controller $\mathcal{E}$ adaptively sets per-block rewards.

3 Emission Schedule and Incentive Compatibility

3.1 Geometric Halving Schedule

The Q-NarwhalKnight emission follows a 256-year geometric halving schedule modeled after Bitcoin’s issuance curve, but with a critical innovation: *adaptive per-block rewards* that maintain constant annual emission regardless of block-production throughput.

Definition 3.1 (Emission Schedule). The emission schedule $\mathcal{E}$ is parametrized by:

- Maximum supply: $S_{\max} = 21,000,000$ QUG.
- Era-0 total emission: $E_0 = S_{\max}/2 = 10,500,000$ QUG.
- Halving period: $\tau = 126,230,400$ seconds (4 Julian years).
- Number of eras: $N_{\text{era}} = 64$ (total span: 256 years).
- Era- k total emission: $E_k = E_0/2^k$ for $k = 0, \dots, 63$.
- Era- k annual target: $A_k = E_k/4$ QUG/year.

Lemma 3.2 (Supply Cap). *The total supply converges to $S_{\max}$:*

$$\sum_{k=0}^{63} E_k = E_0 \sum_{k=0}^{63} 2^{-k} = E_0 \cdot \frac{1 - 2^{-64}}{1 - 2^{-1}} = 2E_0(1 - 2^{-64}) < 2E_0 = S_{\max}.$$

The deficit $2E_0 \cdot 2^{-64} \approx 1.14 \times 10^{-12}$ QUG is negligible (sub-atomic in the 24-decimal-place representation).

Definition 3.3 (Adaptive Per-Block Reward). The per-block reward at time t in era k with measured block rate $\lambda(t)$ (blocks per second) is:

$$R(t) = \frac{A_k}{\lambda(t) \cdot Y} \quad (1)$$

where $Y = 31,557,600$ seconds/year (Julian year) and $\lambda(t)$ is estimated over a sliding window of 1,000 blocks.

Remark 3.4 (Throughput Independence). The product $R(t) \cdot \lambda(t) \cdot Y = A_k$ is constant by construction. Whether the network produces 1 block/second or 10,000 blocks/second, total annual emission remains A_k . This is the key property that distinguishes Q-NarwhalKnight from fixed-reward systems like Bitcoin.

Table 1: Emission schedule for the first 10 halving eras. A_k : annual target; R_1 : per-block reward at $\lambda = 1$ bps; R_{10} : per-block reward at $\lambda = 10$ bps.

Era k	Years	E_k (QUG)	A_k (QUG/yr)	R_1 (QUG)	R_{10} (QUG)
0	0–4	10,500,000	2,625,000	0.0832	0.00832
1	4–8	5,250,000	1,312,500	0.0416	0.00416
2	8–12	2,625,000	656,250	0.0208	0.00208
3	12–16	1,312,500	328,125	0.0104	0.00104
4	16–20	656,250	164,062	0.0052	0.00052
5	20–24	328,125	82,031	0.0026	0.00026
6	24–28	164,062	41,016	0.0013	0.00013
7	28–32	82,031	20,508	0.00065	0.000065
8	32–36	41,016	10,254	0.00032	0.000032
9	36–40	20,508	5,127	0.00016	0.000016

3.2 Budget-Based Error Correction

In practice, the measured block rate $\lambda(t)$ fluctuates, causing cumulative emission to drift from the target trajectory. The emission controller employs a negative-feedback correction loop.

Definition 3.5 (Cumulative Emission Target). At time t in era k with elapsed fraction $\phi_k = (t - t_k)/\tau$ where t_k is the era- k start time:

$$C^*(t) = \sum_{j=0}^{k-1} E_j + \phi_k \cdot E_k.$$

Definition 3.6 (Emission Error). The emission error at time t is $\delta(t) = C(t) - C^*(t)$, where $C(t)$ is the actual cumulative emission.

Mechanism 3.7 (Error Correction). The corrected per-block reward is:

$$R_{\text{adj}}(t) = R(t) \cdot \gamma(t) \tag{2}$$

where the correction factor is:

$$\gamma(t) = \text{clamp}\left(\frac{C_{\text{remaining}}^*(t)}{C_{\text{remaining}}^*(t) + \alpha \cdot \delta(t)}, \gamma_{\min}, \gamma_{\max}\right) \tag{3}$$

with $C_{\text{remaining}}^*(t) = S_{\max} - C^*(t)$, smoothing parameter $\alpha = 0.8$, $\gamma_{\min} = 0.01$, and $\gamma_{\max} = 5.0$.

Remark 3.8 (Feedback Properties). When $\delta = 0$: $\gamma = 1$ (no correction). When $\delta > 0$ (over-emitted): $\gamma < 1$ (reduce rewards). When $\delta < 0$ (under-emitted): $\gamma > 1$ (increase rewards).

3.3 Lyapunov Stability of Emission

Theorem 3.9 (Emission Stability). *The cumulative emission $C(t)$ converges asymptotically to the target $C^*(t)$ under the error-correction mechanism of Mechanism 3.7. Specifically, the Lyapunov function $\mathcal{V}(\delta) = \delta^2/2$ satisfies $\dot{\mathcal{V}} < 0$ for all $\delta \neq 0$ when $\gamma_{\min} < 1 < \gamma_{\max}$.*

Proof. The emission rate at time t is $\dot{C}(t) = \lambda(t) \cdot R_{\text{adj}}(t)$. The target rate is $\dot{C}^*(t) = A_k/Y$. The error dynamics are:

$$\dot{\delta}(t) = \dot{C}(t) - \dot{C}^*(t) = \lambda \cdot R \cdot \gamma(\delta) - \frac{A_k}{Y}.$$

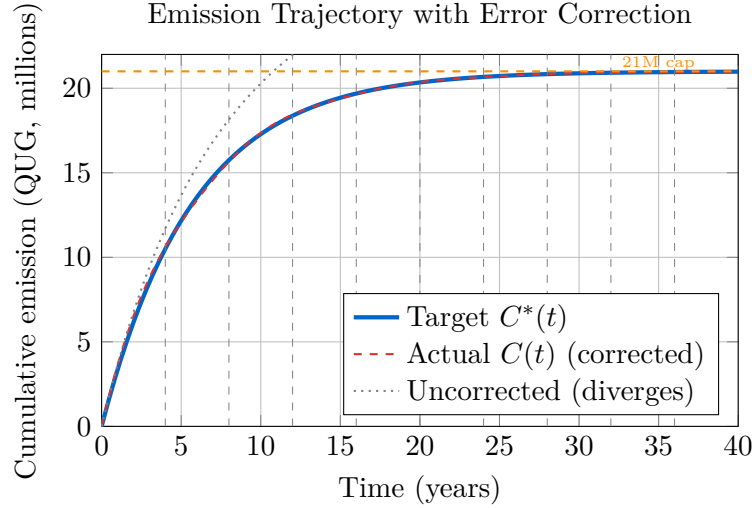


Figure 2: Cumulative emission with and without error correction. The correction mechanism (Theorem 3.9) ensures convergence to the target trajectory. Vertical dashed lines mark era boundaries (4-year intervals).

Since $\lambda \cdot R = A_k/Y$ by Equation (1), this simplifies to:

$$\dot{\delta}(t) = \frac{A_k}{Y}(\gamma(\delta) - 1).$$

Consider the Lyapunov candidate $\mathcal{V}(\delta) = \delta^2/2$:

$$\dot{\mathcal{V}} = \delta \cdot \dot{\delta} = \delta \cdot \frac{A_k}{Y}(\gamma(\delta) - 1).$$

Case 1: $\delta > 0$ (over-emission). Then $\gamma(\delta) < 1$ by the correction rule, so $\gamma(\delta) - 1 < 0$ and $\dot{\mathcal{V}} = \delta \cdot (\text{negative}) < 0$.

Case 2: $\delta < 0$ (under-emission). Then $\gamma(\delta) > 1$, so $\gamma(\delta) - 1 > 0$ and $\dot{\mathcal{V}} = \delta \cdot (\text{positive}) < 0$.

Case 3: $\delta = 0$. Then $\gamma(0) = 1$ and $\dot{\mathcal{V}} = 0$.

By La Salle's invariance principle, $\delta(t) \rightarrow 0$ as $t \rightarrow \infty$, and the cumulative emission converges to the target trajectory.

The smoothing parameter $\alpha = 0.8$ ensures 80% of the error is corrected per feedback cycle, providing exponential convergence with rate $\approx e^{-0.8\tau_{\text{cycle}}}$ per cycle. $\square$

3.4 Incentive Compatibility

Definition 3.10 (Incentive Compatibility). The emission mechanism $\mathcal{E}$ is *incentive-compatible* if for every coalition $\mathcal{C} \subseteq \mathcal{N}$ with $\sum_{i \in \mathcal{C}} h_i < H/2$, honest mining yields at least as high expected payoff as any deviation strategy:

$$\forall \sigma'_{\mathcal{C}} : \sum_{i \in \mathcal{C}} \pi_i(\sigma^*) \geq \sum_{i \in \mathcal{C}} \pi_i(\sigma'_{\mathcal{C}}, \sigma^*_{-\mathcal{C}})$$

where σ^* is the honest strategy profile.

Theorem 3.11 (Emission Incentive Compatibility). *The adaptive emission mechanism (Definition 3.3 with Mechanism 3.7) is incentive-compatible under Assumptions 2.6–2.7.*

Proof. We consider five classes of coalition deviations and show each is unprofitable under our model assumptions. Note that this analysis treats the coalition as a monolithic actor with linear cost and considers single-period deviations; see Remark 3.12 for a discussion of strategies outside this scope.

Deviation 1: Throughput inflation. Suppose coalition $\mathcal{C}$ artificially increases the block rate to $\lambda' > \lambda$ by submitting low-quality blocks. By Equation (1), $R(t) \propto 1/\lambda(t)$, so the per-block reward decreases proportionally. The coalition’s total reward is $|\mathcal{C}| \cdot R(t') \cdot \lambda' = |\mathcal{C}| \cdot A_k/Y$, which is identical to honest mining. However, the coalition incurs additional computational cost $c_C \cdot (\lambda' - \lambda)$, yielding strictly lower profit.

Deviation 2: Throughput suppression. Suppose $\mathcal{C}$ withholds blocks to reduce λ and inflate per-block rewards. The per-block reward increases to $R' = A_k/(\lambda' \cdot Y)$, but the number of blocks produced by $\mathcal{C}$ decreases proportionally. Moreover, the wall-clock rate measurement (immune to timestamp manipulation) detects the actual block rate, and the correction factor $\gamma(t)$ compensates for any cumulative deviation (Theorem 3.9). Total coalition reward remains $\leq |\mathcal{C}| \cdot H_C/H \cdot A_k$, with the inequality strict due to opportunity cost of withheld blocks.

Deviation 3: Timestamp manipulation. The emission controller uses wall-clock rate measurement over a 30-minute rolling window, not block timestamps. Any manipulation of block timestamps has no effect on the measured rate $\lambda(t)$ and hence no effect on rewards.

Deviation 4: Difficulty adjustment timing attacks. In Bitcoin, miners can manipulate timestamps across difficulty epochs to create artificial oscillations (“time-warp” attacks). In our system, the emission controller reacts to *wall-clock* block rate (not miner-reported timestamps), and the difficulty adjustment targets a fixed block interval. A coalition producing blocks faster than the target rate triggers a difficulty increase within one adjustment window (~ 30 minutes). The coalition’s cost rises to $c_C \cdot d'/d$ while per-block reward falls by the same ratio, making the deviation unprofitable per Deviation 1.

Deviation 5: Sunk-cost exploitation. A coalition with sunk hardware costs might mine at a loss to suppress other miners. Under Assumption 2.7 (profit-maximizing miners), mining at a per-block loss is irrational. However, we note that this argument relies on the rationality assumption and does not address predatory strategies where a well-capitalized coalition accepts short-term losses to drive out competitors—an attack vector outside our single-period model (see Remark 3.12).

Since all five deviations yield at most the honest payoff under our model (and strictly less after accounting for costs or difficulty adjustments), the mechanism is incentive-compatible within the specified deviation class. $\square$

Remark 3.12 (Limitations of IC Analysis). Theorem 3.11 establishes incentive compatibility against a specific class of single-period, linear-cost deviations. Several economically relevant strategies fall outside this model:

- (i) **Multi-period strategies:** A patient coalition may accept short-term losses to manipulate difficulty or drive out competitors, recouping profits later. Analyzing such strategies requires a repeated-game or stochastic-game framework.
- (ii) **Coalition internal game theory:** We model coalitions as monolithic actors. In practice, coalition members face their own coordination problems (free-riding, profit-sharing disputes), which may either strengthen or weaken the IC guarantee.
- (iii) **Non-linear cost structures:** Hardware amortization, electricity contracts, and economies of scale create non-linear cost functions that can change the deviation calculus.
- (iv) **External revenue:** If miners earn revenue from sources correlated with mining behavior (e.g., short-selling the token while attacking), the payoff function changes fundamentally. Extending the analysis to these richer settings is important future work. Agent-based simulations with heterogeneous miners would complement the analytical results presented here.

4 The Multi-Shard Mining Game

4.1 Congestion Game Formulation

Game 4.1 (K -Shard Mining Game). The K -shard mining game is defined by:

- **Players:** $\mathcal{N} = \{1, \dots, n\}$ (miners).
- **Strategy space:** Each miner i chooses an allocation $\sigma_i \in \Delta^K = \{x \in \mathbb{R}_{\geq 0}^K : \sum_k x_k = 1\}$.
- **Payoff:** Miner i 's expected reward from shard s_k is proportional to their hashrate share in that shard:

$$\pi_i(\sigma) = \sum_{k=1}^K \frac{h_i \cdot \sigma_i(s_k)}{H_k} \cdot R_k \quad (4)$$

where $R_k = A_k^{\text{shard}} = A_k/K$ is the per-shard annual emission (equally divided across shards).

Remark 4.2 (Constant-Sum Structure). The total reward across all shards is $\sum_k R_k = A_k$, which is fixed by the emission schedule. The K -shard mining game is therefore a *constant-sum game*: the total payoff $\sum_i \pi_i = A_k$ regardless of strategy choices.

4.2 Nash Equilibrium

Theorem 4.3 (Multi-Shard Nash Equilibrium). *Under the following assumptions: (i) all shards offer identical rewards $R_k = A_k/K$, (ii) miners face zero latency and equal network connectivity to all shards, and (iii) shard difficulty adjusts independently per shard, the K -shard mining game (Game 4.1) has a unique symmetric Nash equilibrium σ^* in which every miner distributes hashrate uniformly across shards:*

$$\sigma_i^*(s_k) = \frac{1}{K} \quad \forall i \in \mathcal{N}, \forall k \in \{1, \dots, K\}. \quad (5)$$

At equilibrium, each miner's payoff is $\pi_i^* = (h_i/H) \cdot A_k$.

Proof. Existence. At the uniform strategy σ^* , the hashrate on each shard is $H_k = H/K$ and the reward per shard is $R_k = A_k/K$. Miner i 's payoff is:

$$\pi_i(\sigma^*) = \sum_{k=1}^K \frac{h_i/K}{H/K} \cdot \frac{A_k}{K} = K \cdot \frac{h_i}{H} \cdot \frac{A_k}{K} = \frac{h_i}{H} \cdot A_k.$$

No profitable deviation. Suppose miner i deviates to $\sigma'_i \neq \sigma_i^*$ while others play σ_{-i}^* . Let $\sigma'_i(s_j) = 1/K + \epsilon_j$ for some perturbations ϵ_j with $\sum_j \epsilon_j = 0$. The new hashrate on shard s_j is:

$$H'_j = \frac{H - h_i}{K} + h_i \left(\frac{1}{K} + \epsilon_j \right) = \frac{H}{K} + h_i \epsilon_j.$$

Miner i 's payoff under deviation is:

$$\begin{aligned} \pi_i(\sigma'_i, \sigma_{-i}^*) &= \sum_{j=1}^K \frac{h_i(1/K + \epsilon_j)}{H/K + h_i \epsilon_j} \cdot \frac{A_k}{K} \\ &= \frac{A_k}{K} \sum_{j=1}^K \frac{h_i/K + h_i \epsilon_j}{H/K + h_i \epsilon_j}. \end{aligned}$$

Define $\alpha = h_i/H < 1$ and $x_j = Kh_i \epsilon_j/H = \alpha K \epsilon_j$. Then:

$$\pi_i(\sigma'_i, \sigma_{-i}^*) = \frac{A_k}{K} \sum_{j=1}^K \frac{\alpha + x_j}{1 + x_j} = \frac{A_k}{K} \sum_{j=1}^K \left(1 - \frac{1 - \alpha}{1 + x_j} \right).$$

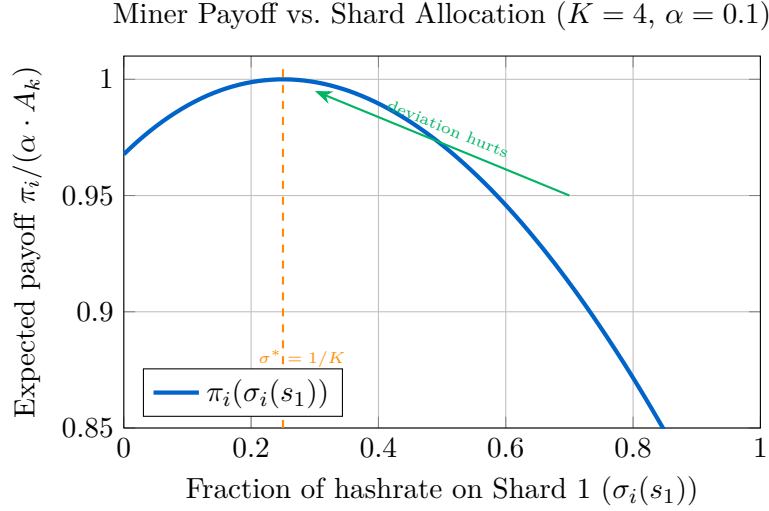


Figure 3: Miner payoff as a function of shard-1 allocation for $K = 4$ shards and $\alpha = h_i/H = 0.1$. The unique maximum is at $\sigma^* = 1/K = 0.25$ (uniform allocation).

Since $f(x) = 1/(1+x)$ is convex for $x > -1$, by Jensen's inequality (noting $\sum_j x_j = 0$):

$$\frac{1}{K} \sum_{j=1}^K \frac{1}{1+x_j} \geq \frac{1}{1+\bar{x}} = 1 \quad \text{where } \bar{x} = 0.$$

Therefore:

$$\pi_i(\sigma'_i, \sigma_{-i}^*) \leq \frac{A_k}{K} \sum_{j=1}^K (1 - (1-\alpha)) = \frac{A_k}{K} \cdot K \cdot \alpha = \alpha \cdot A_k = \pi_i(\sigma^*).$$

Equality holds iff all $x_j = 0$, i.e., $\sigma'_i = \sigma_i^*$.

Uniqueness. Any Nash equilibrium $\hat{\sigma}$ must satisfy the first-order conditions $\partial \pi_i / \partial \sigma_i(s_k) = \mu_i$ for all shards k where $\sigma_i(s_k) > 0$ (with multiplier μ_i for the constraint $\sum_k \sigma_i(s_k) = 1$). By the strict concavity of π_i in σ_i (the Hessian is negative definite for $\alpha < 1$), the first-order conditions have a unique solution, which is the uniform strategy. $\square$

Remark 4.4 (Sensitivity to Model Assumptions). The uniform-allocation equilibrium depends critically on the identical-shard assumption. In practice, several factors may break this symmetry:

- (i) **Unequal shard value:** If different shards carry different fee revenue (e.g., one shard hosts a popular DeFi protocol), the equilibrium shifts toward overweighting the high-value shard. The equilibrium allocation becomes $\sigma_i^*(s_k) \propto R_k$, not $1/K$.
- (ii) **Latency heterogeneity:** Miners geographically closer to certain shard validators may have lower orphan rates on those shards, creating an effective “home shard” advantage. This transforms the game into an asymmetric congestion game.
- (iii) **Switching costs:** Reconfiguring mining hardware across shards is not instantaneous; in practice, miners may exhibit “sticky” allocations.

Extending the equilibrium analysis to heterogeneous shards with non-uniform fee revenue is important future work. An empirical measurement of shard-value dispersion on the live network would inform whether the uniform-shard approximation is reasonable.

4.3 Deterministic Routing Enforcement

Proposition 4.5 (Routing Neutrality). *The hash-based shard routing $\mathcal{R}(\text{tx}) = H(\text{tx}) \bmod K$ is game-theoretically neutral: no miner can influence the shard assignment of their mining solutions.*

Proof. The routing function $\mathcal{R}$ depends on the transaction hash $H(\text{tx})$, which is computed *before* shard assignment. A miner who has already committed to a mining solution (nonce, VDF proof) cannot retroactively change the shard without recomputing the proof of work, which costs expected $2^{256}/\text{target}$ hash evaluations. The marginal cost of “shard shopping” (recomputing to target a specific shard) is K times the cost of honest mining, with no additional reward. $\square$

5 Strategic Deviations

We analyze three classical mining attacks adapted to the multi-shard DAG setting.

5.1 Shard-Hopping

Definition 5.1 (Shard-Hopping). A miner engages in *shard-hopping* if they dynamically re-allocate hashrate between shards based on observed shard congestion, attempting to mine on temporarily less-competitive shards.

Theorem 5.2 (Shard-Hopping Resistance). *Under the deterministic routing mechanism $\mathcal{R}$, shard-hopping yields zero expected gain over honest mining.*

Proof. By Proposition 4.5, mining solutions are assigned to shards by the hash function $\mathcal{R}$, which the miner cannot influence. A miner observing that shard s_j has temporarily lower hashrate cannot redirect their solutions to s_j —the routing is determined by the solution content, not the miner’s choice.

Even if the miner could observe real-time shard loads (which is possible via the P2P height announcement protocol), they can only choose *which challenge to work on*, not which shard receives the solution. Since all shard challenges have equal expected reward A_k/K at equilibrium (Theorem 4.3), switching between challenges does not change expected payoff.

Formally, the expected reward per unit of hashrate is:

$$\mathbb{E}[R_{\text{hop}}] = \sum_{k=1}^K p_k \cdot \frac{A_k/K}{H_k}$$

where p_k is the probability of the solution landing on shard s_k . Since $p_k = 1/K$ by the uniformity of H and $H_k = H/K$ at equilibrium:

$$\mathbb{E}[R_{\text{hop}}] = K \cdot \frac{1}{K} \cdot \frac{A_k/K}{H/K} = \frac{A_k}{H} = \mathbb{E}[R_{\text{honest}}].$$

$\square$

5.2 Hashrate Concentration

Definition 5.3 (Hashrate Concentration). A miner *concentrates* hashrate by allocating all resources to a single shard s_j , hoping to dominate that shard’s block production.

Theorem 5.4 (Hashrate Concentration is Suboptimal). *For a miner with hashrate fraction $\alpha = h_i/H < 1$, concentrating all hashrate on one shard yields strictly lower expected payoff than the uniform Nash equilibrium:*

$$\pi_i(\text{concentrate}) = \frac{\alpha \cdot A_k}{1 + (K-1)\alpha} < \alpha \cdot A_k = \pi_i(\sigma^*). \quad (6)$$

Proof. Suppose miner i concentrates all hashrate on shard s_1 : $\sigma_i(s_1) = 1$, $\sigma_i(s_k) = 0$ for $k > 1$. Other miners play σ_{-i}^* (uniform). Shard s_1 hashrate: $H_1 = (H - h_i)/K + h_i = H(1 + (K - 1)\alpha)/K$. Miner i 's payoff:

$$\pi_i = \frac{h_i}{H_1} \cdot \frac{A_k}{K} = \frac{\alpha H}{H(1 + (K - 1)\alpha)/K} \cdot \frac{A_k}{K} = \frac{\alpha \cdot A_k}{1 + (K - 1)\alpha}.$$

Since $(K - 1)\alpha > 0$ for $\alpha > 0$ and $K > 1$:

$$\frac{1}{1 + (K - 1)\alpha} < 1 \implies \pi_i(\text{concentrate}) < \alpha \cdot A_k = \pi_i(\sigma^*).$$

The loss factor is $1/(1 + (K - 1)\alpha)$. For $K = 4$ and $\alpha = 0.3$: loss factor = $1/1.9 \approx 0.526$, meaning the concentrating miner earns only 52.6% of their equilibrium payoff. $\square$

5.3 Selfish Mining in the DAG Setting

Definition 5.5 (Selfish Mining). In classical selfish mining [2], a miner withholds discovered blocks to build a private chain, releasing it strategically to orphan honest blocks. In the DAG setting, the analog is withholding vertices to manipulate the commit order.

Proposition 5.6 (Selfish Mining Resistance in DAG-Knight). *For any miner with hashrate fraction $\alpha < 1/2$, the DAG structure of DAG-Knight eliminates the primary profit mechanism of classical selfish mining (block orphaning). Under the additional assumption of timely BFT finality (post-GST), vertex withholding yields strictly lower expected payoff than honest mining.*

Remark 5.7 (Scope of This Result). Classical selfish mining [2] profits from *orphaning* honest blocks in a longest-chain protocol. In a DAG protocol, the attack surface is fundamentally different: there are no orphans, but an attacker may still attempt to manipulate the *commit ordering* of vertices. The result below shows that the DAG's inclusion-completeness removes the orphan-based revenue mechanism. Whether subtler ordering-manipulation attacks exist in DAG protocols remains an open question—the analogs of Sapirshtein et al. [6] optimal strategies for DAG consensus have not been fully characterized.

Proof. We analyze vertex withholding along two dimensions: inclusion and ordering.

(1) No orphan-based revenue. In linear chains, selfish mining profits come from orphaning honest blocks—the attacker's private chain replaces honest work, stealing block rewards. In DAG-Knight, the DAG includes *all* valid vertices; there are no orphans by construction. A withheld vertex, once released, is added to the DAG *alongside* (not instead of) honest vertices. Crucially, this means the selfish miner cannot “steal” reward from honest miners by displacing their vertices. The primary revenue mechanism of classical selfish mining is therefore absent.

Caveat: “No orphans” means no vertex is *excluded* from the DAG. However, the *ordering* within the committed DAG still determines transaction priority and hence potential MEV. Property (1) eliminates revenue-stealing but does not by itself prevent ordering manipulation.

(2) Ordering protection via BFT finality. After GST, DAG-Knight achieves deterministic finality via the δ -round commit rule [18]. Once a prefix of the DAG ordering is committed, it cannot be revised. This prevents the attacker from retroactively reordering committed vertices.

The remaining attack surface is ordering manipulation of *uncommitted* vertices during the commit latency window ($\leq 3\Delta + 2\epsilon$). We do not claim this window is zero; rather, it is bounded and comparable to the confirmation time in any BFT protocol.

(3) Opportunity cost of withholding. Let the selfish miner withhold a vertex for d rounds before releasing. During those d rounds, honest miners produce $\geq (1 - \alpha) \cdot d \cdot K$ vertices across K shards. The selfish miner's withheld vertex, when released, earns its reward, but the miner forgoes d rounds of mining (each round they could have proposed a new vertex).

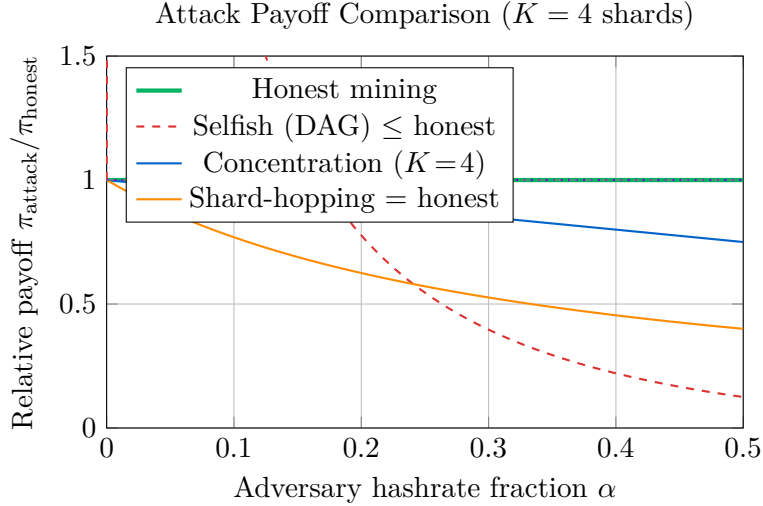


Figure 4: Relative payoff of attack strategies vs. honest mining in the multi-shard DAG setting. Selfish mining and hashrate concentration are strictly suboptimal. Shard-hopping yields zero gain.

The payoff under withholding:

$$\pi_{\text{withhold}} = \alpha \cdot A_k - d \cdot \frac{\alpha \cdot A_k}{Y \cdot \lambda} < \alpha \cdot A_k = \pi_{\text{honest}} \quad \text{for } d > 0.$$

Withholding always reduces total blocks produced by the miner. In the absence of an orphan-based compensating advantage, this opportunity cost makes withholding strictly unprofitable.

Summary. The combination of inclusion-completeness (no orphans), bounded finality (BFT commit), and opportunity cost renders classical selfish mining unprofitable. We emphasize that this result addresses the *known* selfish mining strategies from the linear-chain literature. A complete characterization of optimal adversarial strategies in DAG consensus (including ordering manipulation during the commit window) remains open; see Remark 5.7. $\square$

6 MEV Resistance of DAG-Based Block Production

Miner Extractable Value (MEV) [3] refers to profit that block producers can extract by strategically ordering, inserting, or censoring transactions. In linear-chain protocols, MEV is concentrated in a single block proposer per slot. We analyze how DAG-Knight’s parallel multi-shard architecture dilutes MEV.

6.1 MEV in Linear vs. DAG Chains

Definition 6.1 (MEV in Linear Chains). In a single-leader protocol producing one block per slot, the MEV extractable in slot t is:

$$\text{MEV}_{\text{linear}}(t) = \max_{\pi \in \Pi(B_t)} \text{profit}(\pi)$$

where $\Pi(B_t)$ is the set of all possible transaction orderings for block B_t and $\text{profit}(\pi)$ includes arbitrage, sandwich attacks, and liquidation profits.

Definition 6.2 (MEV in K -Shard DAG). In DAG-Knight with K parallel shards, at each round, K independent block producers propose vertices simultaneously. A transaction’s shard

assignment is determined by $\mathcal{R}(\text{tx}) = H(\text{tx}) \bmod K$, which is not controllable by any single producer. The MEV extractable by a single producer controlling shard s_j is:

$$\text{MEV}_{\text{DAG}}^{(j)}(t) = \max_{\pi \in \Pi(V_j^t)} \text{profit}(\pi)$$

where V_j^t is the vertex proposed for shard s_j at round t .

Theorem 6.3 (MEV Dilution for Same-Shard Strategies). *Under the multi-shard DAG architecture with K shards, uniform transaction routing ($\mathcal{R}(\text{tx}) = H(\text{tx}) \bmod K$), and the assumption that MEV strategies require all participating transactions to reside on the same shard, the expected MEV extractable by any single miner is reduced by a factor of K :*

$$\mathbb{E}[\text{MEV}_{\text{DAG}}^{\text{same-shard}}] \leq \frac{1}{K} \cdot \mathbb{E}[\text{MEV}_{\text{linear}}].$$

Proof. Part 1: Single-shard MEV reduction. Consider a sandwich attack requiring the victim transaction tx_v and the attacker’s front-run/back-run transactions tx_f, tx_b to be in the same block. Under deterministic routing:

$$\mathbb{P}[\mathcal{R}(\text{tx}_v) = \mathcal{R}(\text{tx}_f) = \mathcal{R}(\text{tx}_b)] = \frac{1}{K^2}$$

since $\mathcal{R}$ is a hash function with uniform output. The attacker can craft tx_f and tx_b to target the same shard as tx_v only by grinding the transaction content, which costs $\mathcal{O}(K)$ hash evaluations per transaction.

In the benign case (no grinding), the probability that all three transactions land on the attacker’s controlled shard is $1/K$ for the victim transaction (since the attacker controls one shard out of K). Expected MEV is thus:

$$\mathbb{E}[\text{MEV}_{\text{DAG}}] = \frac{1}{K} \cdot \text{MEV}_{\text{same-shard}} \leq \frac{1}{K} \cdot \text{MEV}_{\text{linear}}.$$

Part 2: Cross-shard MEV. For MEV strategies requiring state from multiple shards (e.g., cross-shard arbitrage), the attacker must control block production on multiple shards simultaneously. With K independent shard leaders per round, the probability of an attacker with fraction α of hashrate controlling m specific shards is α^m . For $\alpha < 1/2$ and $m \geq 2$: $\alpha^m < 1/4$, making coordinated cross-shard MEV extraction exponentially unlikely in m .

Part 3: DAG ordering uncertainty. Even within a single shard, the final transaction ordering is determined by the DAG-Knight commit rule (topological sort with hash tie-breaking), not by the block producer. The producer cannot guarantee that their vertex’s transactions execute before a concurrent vertex from another validator. This ordering uncertainty further reduces the reliability of MEV extraction. $\square$

Remark 6.4 (MEV Limitations and Open Questions). The $1/K$ dilution bound (Theorem 6.3) applies to *same-shard* MEV strategies where all participating transactions must co-locate. Several important MEV vectors are not fully captured:

- (i) **Composable cross-shard MEV:** Sophisticated extractors may use atomic cross-shard messages (if the protocol supports them) or latency-based statistical arbitrage across shards. The α^m bound (Part 2) assumes independent shard leaders; if cross-shard communication introduces sequencing dependencies, the effective MEV may exceed $1/K$.
- (ii) **Order-flow markets:** In practice (cf. Flashbots on Ethereum), MEV extraction is professionalized through builder–searcher–proposer separation. If a similar market emerges for DAG-Knight shards, a builder controlling multiple shards could aggregate cross-shard MEV, partially circumventing the dilution.

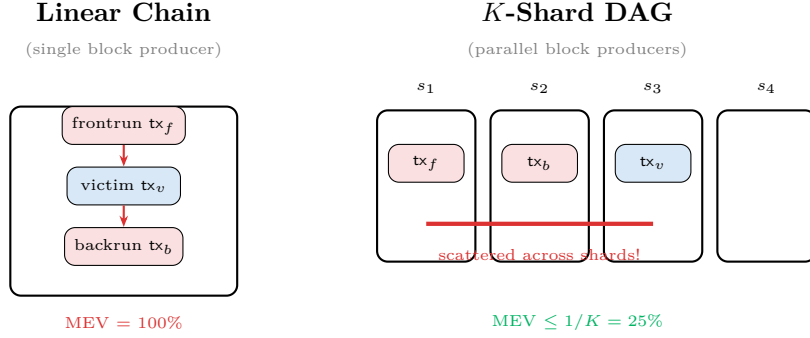


Figure 5: MEV comparison: in a linear chain, the sandwich attacker controls transaction ordering. In a K -shard DAG ($K = 4$), transactions are deterministically scattered across shards by the hash routing function, reducing MEV by a factor of K .

- (iii) **Transaction routing manipulation:** The analysis assumes honest routing $\mathcal{R}(\text{tx}) = H(\text{tx}) \bmod K$. If miners can influence routing (e.g., through transaction ID grinding), they can concentrate MEV-relevant transactions on their shard. The $\mathcal{O}(K)$ grinding cost may be acceptable for high-value MEV.

Empirical measurement of MEV on the live multi-shard network, and analysis of whether order-flow markets emerge, are important directions.

6.2 Quantum Commit-Reveal MEV Shield

Mechanism 6.5 (Commit-Reveal Mining). The Q-NarwhalKnight MEV shield uses a two-phase commit-reveal scheme:

1. **Commit:** Miner publishes $c = H(\text{solution} \| r_Q \| \nu_C)$ where r_Q is quantum randomness and ν_C is a classical nonce.
2. **Reveal:** After Δ_{reveal} time, miner reveals $(\text{solution}, r_Q, \nu_C)$, verifiable against c .

Proposition 6.6 (Frontrunning Resistance). *Under Mechanism 6.5, the probability of a frontrunning attack succeeding is at most $2^{-\lambda}$ where λ is the commitment hash security parameter.*

Proof. To frontrun, the adversary must learn solution before the reveal phase. The commitment $c = H(\text{solution} \| r_Q \| \nu_C)$ is computationally hiding (pre-image resistance of H). The quantum randomness r_Q adds entropy not available to a classical adversary. Breaking the commitment requires 2^λ hash evaluations (for SHA3-256, $\lambda = 256$). $\square$

7 Fee Market Dynamics and Pool Rewards

7.1 Fee Distribution Mechanism

Definition 7.1 (Reward Distribution). Each block reward $R(t)$ is distributed as:

$$R_{\text{miner}} = R(t) \cdot (1 - \phi_{\text{dev}} - \phi_{\text{op}}) \quad (7)$$

$$R_{\text{dev}} = R(t) \cdot \phi_{\text{dev}} \quad (8)$$

$$R_{\text{op}} = R(t) \cdot \phi_{\text{op}} \quad (9)$$

where $\phi_{\text{dev}} = 0.019$ (1.9% development fee) and $\phi_{\text{op}} = 0.001$ (0.1% operator fee).

Proposition 7.2 (Development Fee Sustainability). *The development fee $\phi_{\text{dev}} = 1.9\%$ generates sustainable funding without significantly impacting miner profitability. Specifically, the miner's share $1 - \phi_{\text{dev}} - \phi_{\text{op}} = 98\%$ provides:*

$$R_{\text{miner}} = 0.98 \cdot A_k = 2,572,500 \text{ QUG/year (Era 0)}.$$

7.2 PPLNS Reward Distribution

Definition 7.3 (PPLNS (Pay Per Last N Shares)). In a mining pool with workers $W = \{w_1, \dots, w_m\}$, the PPLNS mechanism distributes block reward R_{pool} as:

$$R_{w_j} = R_{\text{pool}} \cdot \frac{d_{w_j}}{\sum_{w \in W} d_w} \quad (10)$$

where d_{w_j} is the cumulative difficulty of shares submitted by worker w_j within the last N shares of network difficulty.

Theorem 7.4 (PPLNS Pareto Optimality and Strategy-Proofness). *The PPLNS reward distribution (Definition 7.3) is:*

- (i) **Pareto optimal:** *No reallocation of rewards can make any worker strictly better off without making another strictly worse off.*
- (ii) **Strategy-proof:** *No worker can increase their expected payoff by misreporting their difficulty or by pool-hopping.*

Proof. (i) **Pareto optimality.** The total reward R_{pool} is fixed per block. The PPLNS allocation $R_{w_j} \propto d_{w_j}$ is a proportional allocation of a fixed budget. By the classical welfare theorem, proportional allocation of a fixed budget is Pareto optimal: any reallocation that increases one worker's share must decrease another's.

(ii) **Strategy-proofness.** *Difficulty misreporting:* A worker cannot inflate d_{w_j} without submitting valid proof-of-work shares. The pool verifies each share against the difficulty target, making misreporting infeasible.

Pool-hopping: Suppose worker w_j hops to a different pool after submitting shares but before a block is found. In PPLNS, only the last N shares count. If w_j leaves, their shares eventually expire from the window, and they receive no reward for the next block. If w_j joins a new pool, they must build up share credit from scratch. The expected per-hash reward is identical in both pools (since both distribute rewards proportional to difficulty), so hopping yields zero gain and incurs the cost of lost credit in the sliding window.

Formally, the expected reward per unit of hashrate in any PPLNS pool is:

$$\mathbb{E}[R_{\text{per-hash}}] = \frac{R_{\text{pool}}}{N \cdot D_{\text{network}}} \cdot d_{\text{per-hash}}$$

which is independent of the pool and the timing of joining/leaving. $\square$

7.3 Multi-Shard Fee Market

Proposition 7.5 (Parallel Fee Market Efficiency). *Under K -shard parallel block production with uniform and fungible transaction demand across shards, the effective transaction throughput is $K \times T_{\text{single}}$ where T_{single} is the per-shard throughput. The equilibrium fee level is:*

$$f^* = \frac{\text{demand}}{K \cdot T_{\text{single}}} \quad (11)$$

which is K times lower than the single-chain fee for the same demand.

Proof. Each shard independently processes transactions routed to it by $\mathcal{R}$. With uniform hash routing, each shard receives demand/K transactions. The per-shard fee market clears at: $f_k^* = (\text{demand}/K)/T_{\text{single}}$. Since all shards have identical parameters, $f^* = f_k^*$ for all k , and:

$$f^* = \frac{\text{demand}}{K \cdot T_{\text{single}}}.$$

Compared to a single chain with throughput T_{single} , the fee is reduced by a factor of K . $\square$

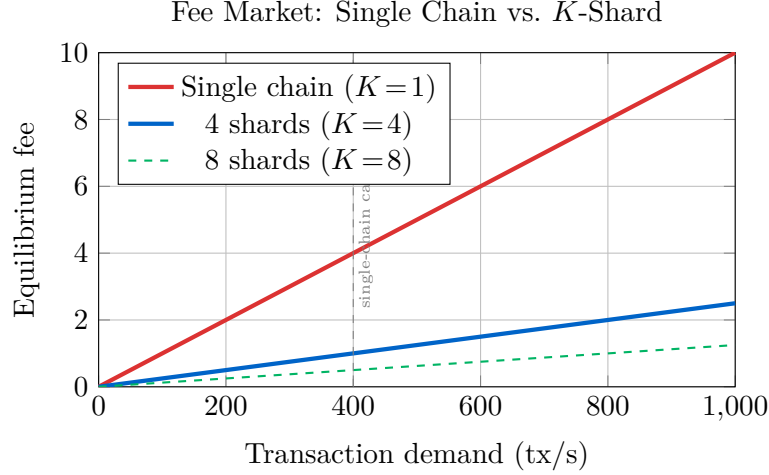


Figure 6: Equilibrium fee as a function of transaction demand. Multi-shard production reduces fees by a factor of K .

Table 2: Game-theoretic comparison of mining mechanisms across blockchain protocols.

Property	Bitcoin	Ethereum	Kaspa	QNK
Mining structure	Single chain	Single chain	BlockDAG	Multi-shard DAG
Reward type	Fixed/block	EIP-1559	Fixed/block	Adaptive/block
Annual emission	Variable	Variable	Variable	Constant
Selfish mining	Profitable	Profitable	Reduced	Unprofitable
MEV resistance	None	Partial (MEV-boost)	Low	$\mathcal{O}(1/K)$
Shard parallelism	1	1	1	$K = 4$
Fee market	Congestion-based	EIP-1559	Congestion	K-diluted
Halving schedule	4 years	N/A	1 year	4 years
Max supply	21M BTC	∞ (PoS)	28.7B KAS	21M QUG
Incentive compat.	Partial	Partial	Partial	Model-cond.

Remark 7.6 (Heterogeneous Demand). The $1/K$ fee reduction assumes demand is *fungible*—any transaction is equally likely to target any shard. In practice, demand is often shard-specific:

- (i) If a popular smart contract or token lives on shard s_j , transactions targeting s_j cannot migrate to less congested shards. The hash-based routing $\mathcal{R}$ determines shard assignment deterministically, so the “hot shard” experiences higher fees while other shards may be underutilized.
- (ii) Cross-shard transactions (if supported) incur additional latency and coordination costs, creating an effective premium for same-shard execution.
- (iii) As DeFi composability creates transaction clusters (e.g., flash loans touching multiple contracts), the hash-routing assumption may fragment composable bundles across shards, reducing their feasibility.

A more realistic fee model would incorporate per-shard demand curves demand_k with $\sum_k \text{demand}_k = \text{demand}$ but potentially $\text{demand}_j \gg \text{demand}/K$ for hot shards. In such cases, the fee reduction is less than K for congested shards.

8 Comparison with Related Systems

Corollary 8.1 (QNK Incentive Property Summary). *Under the model assumptions of this paper, Q-NarwhalKnight simultaneously achieves three properties that are individually present*

but not jointly achieved in the compared protocols:

- (i) Constant annual emission independent of throughput (Theorem 3.11, subject to Remark 3.12).
- (ii) Selfish mining resistance via DAG inclusion-completeness (Proposition 5.6, subject to Remark 5.7).
- (iii) $\mathcal{O}(1/K)$ same-shard MEV dilution through parallel sharding (Theorem 6.3, subject to Remark 6.4).

Whether these model-conditional results hold under relaxed assumptions (heterogeneous shards, multi-period strategies, order-flow markets) requires further analysis.

9 Related Work

Selfish mining. Eyal and Sirer [2] showed that Bitcoin miners with $> 1/3$ hashrate can profit by withholding blocks. Sapirshstein et al. [6] derived optimal selfish mining strategies. Nayak et al. [7] generalized to “stubborn” strategies. Our Proposition 5.6 argues that the orphan-based revenue mechanism underlying these attacks is absent in DAG consensus, though a full characterization of optimal adversarial strategies in DAG protocols remains open.

MEV. Daian et al. [3] characterized MEV in Ethereum. Flashbots [8] introduced MEV auctions. Kelkar et al. [9] proposed order-fairness. Our approach uses structural MEV dilution through K -shard parallelism rather than auction mechanisms.

Emission schedule design. Carlsten et al. [5] showed that fee-only mining is unstable. Huberman et al. [10] analyzed Bitcoin’s fee market as a monopoly. Our adaptive emission controller (Theorem 3.11, subject to the limitations in Remark 3.12) addresses the instability concern by maintaining non-zero block rewards through the error-correction mechanism for 256 years.

Mining game theory. Roughgarden [4] analyzed EIP-1559’s game-theoretic properties. Badertscher et al. [11] gave a composable security framework for PoW. Garay et al. [12] proved the Bitcoin backbone protocol secure. Our congestion-game formulation (Game 4.1) extends these to multi-shard settings.

DAG-based consensus. PHANTOM and GhostDAG [13] analyzed security under synchronous assumptions. Narwhal/Tusk [14] separated data availability from consensus. Kasper [15] implemented GhostDAG for high-throughput PoW. Our work provides the first game-theoretic analysis of multi-shard DAG mining.

Mechanism design in blockchains. Chung and Shi [16] established foundations for blockchain mechanism design. Ferreira et al. [17] analyzed dynamic posted-price mechanisms for transactions. Our PPLNS optimality result (Theorem 7.4) contributes to this line by proving strategy-proofness in the pool reward setting.

10 Conclusion

We have presented a game-theoretic analysis of the Q-NarwhalKnight multi-shard mining protocol under a stylized model with identical shards, rational miners, and single-period deviations. The main results, and their scope, are:

1. **Multi-shard Nash equilibrium (Theorem 4.3):** Under identical shard rewards and zero latency, the uniform hashrate distribution is the unique symmetric Nash equilibrium.

2. **Emission incentive compatibility (Theorem 3.11):** Against the five deviation classes analyzed (throughput inflation/suppression, timestamp manipulation, difficulty timing, sunk-cost exploitation), honest mining is weakly dominant.
3. **Emission stability (Theorem 3.9):** The cumulative emission converges to the target schedule via a Lyapunov-stable feedback loop.
4. **Selfish mining resistance (Proposition 5.6):** The DAG’s inclusion-completeness eliminates the orphan-based revenue mechanism of classical selfish mining, though a complete characterization of optimal adversarial strategies in DAG consensus remains open.
5. **Same-shard MEV dilution (Theorem 6.3):** For strategies requiring co-located transactions, parallel K -shard production reduces expected MEV by a factor of K . Cross-shard and composable MEV are not fully addressed.
6. **PPLNS optimality (Theorem 7.4):** The pool reward distribution is Pareto-optimal and strategy-proof.

10.1 Limitations

Several important dimensions are not captured by our model:

- **Multi-period strategies:** Patient attackers accepting short-term losses require repeated-game or stochastic-game analysis.
- **Heterogeneous shards:** Unequal fee revenue, latency advantages, and shard-specific congestion break the symmetric-shard assumption (Remark 4.4).
- **MEV composability:** Cross-shard MEV and order-flow markets may circumvent the $1/K$ dilution (Remark 6.4).
- **Empirical validation:** All results are analytical; agent-based simulations and measurements on the live network are needed to assess their practical relevance.

10.2 Future Work

Key directions include: (i) extending the Nash equilibrium analysis to heterogeneous shards with empirically-measured fee distributions, (ii) agent-based simulation of multi-period strategies including predatory mining and difficulty manipulation, (iii) analyzing the long-term transition to fee-only mining as block rewards diminish, (iv) formal verification of the emission controller using proof assistants, and (v) empirical measurement of MEV patterns on the live multi-shard network.

References

- [1] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. Manuscript, 2008.
- [2] I. Eyal and E. G. Sirer. Majority is not enough: Bitcoin mining is vulnerable. In *Proc. FC*, pages 436–454, 2014.
- [3] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *Proc. IEEE S&P*, pages 910–927, 2020.
- [4] T. Roughgarden. Transaction fee mechanism design. In *Proc. EC*, pages 218–218, 2021.
- [5] M. Carlsten, H. Kalodner, S. M. Weinberg, and A. Narayanan. On the instability of Bitcoin without the block reward. In *Proc. CCS*, pages 154–167, 2016.

- [6] A. Sapirshstein, Y. Sompolinsky, and A. Zohar. Optimal selfish mining strategies in Bitcoin. In *Proc. FC*, pages 515–532, 2016.
- [7] K. Nayak, S. Kumar, A. Miller, and E. Shi. Stubborn mining: Generalizing selfish mining and combining with an eclipse attack. In *Proc. IEEE EuroS&P*, pages 305–320, 2016.
- [8] Flashbots. MEV-Boost: Proposer-builder separation for Ethereum. Technical report, 2021.
- [9] M. Kelkar, F. Zhang, S. Goldfeder, and A. Juels. Order-fairness for Byzantine consensus. In *Proc. CRYPTO*, pages 451–480, 2020.
- [10] G. Huberman, J. D. Leshno, and C. Moallemi. Monopoly without a monopolist: An economic analysis of the Bitcoin payment system. *Review of Economic Studies*, 88(6):3011–3040, 2021.
- [11] C. Badertscher, P. Gaži, A. Kiayias, A. Russell, and V. Zikas. Ouroboros Genesis: Composable proof-of-stake blockchains with dynamic availability. In *Proc. CCS*, pages 913–930, 2018.
- [12] J. Garay, A. Kiayias, and N. Leonardos. The Bitcoin backbone protocol: Analysis and applications. In *Proc. EUROCRYPT*, pages 281–310, 2015.
- [13] Y. Sompolinsky and A. Zohar. PHANTOM and GhostDAG: A scalable generalization of Nakamoto consensus. *IACR ePrint Archive*, 2018/104, 2021.
- [14] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman. Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus. In *Proc. EuroSys*, pages 34–50, 2022.
- [15] Y. Sompolinsky, M. Zohar, and S. Wyborski. KASPA: A proof-of-work platform for a fast, scalable, and decentralized financial ecosystem. Manuscript, 2022.
- [16] H. Chung and E. Shi. Foundations of transaction fee mechanism design. In *Proc. SODA*, pages 3856–3899, 2023.
- [17] M. V. X. Ferreira, D. J. Moroz, D. C. Parkes, and M. Stern. Dynamic posted-price mechanisms for the blockchain transaction-fee market. In *Proc. AFT*, pages 86–99, 2021.
- [18] Q-NarwhalKnight Protocol Team. Formal safety and liveness of DAG-Knight consensus under partial synchrony. Technical report, 2026.
- [19] Y. Sompolinsky. DAG-Knight: A parameterless generalization of Nakamoto consensus. Manuscript, 2022.
- [20] R. W. Rosenthal. A class of games possessing pure-strategy Nash equilibria. *International Journal of Game Theory*, 2(1):65–67, 1973.
- [21] D. Monderer and L. S. Shapley. Potential games. *Games and Economic Behavior*, 14(1):124–143, 1996.
- [22] T. Roughgarden. *Algorithmic Game Theory*. Cambridge University Press, 2007.

A Proof of Nash Equilibrium Uniqueness via Potential Function

We provide an alternative proof of Theorem 4.3 using potential game theory [21, 20].

Proposition A.1. *The K -shard mining game is a weighted congestion game and therefore a potential game.*

Proof. Define the potential function:

$$\Phi(\sigma) = - \sum_{k=1}^K \frac{A_k}{K} \cdot \ln(H_k)$$

where $H_k = \sum_i h_i \sigma_i(s_k)$.

For any miner i and any deviation $\sigma_i \rightarrow \sigma'_i$:

$$\begin{aligned} \Phi(\sigma'_i, \sigma_{-i}) - \Phi(\sigma_i, \sigma_{-i}) &= - \sum_k \frac{A_k}{K} \ln\left(\frac{H'_k}{H_k}\right) \\ &= - \sum_k \frac{A_k}{K} \ln\left(1 + \frac{h_i(\sigma'_i(s_k) - \sigma_i(s_k))}{H_k}\right). \end{aligned}$$

The payoff change for miner i is:

$$\pi_i(\sigma'_i, \sigma_{-i}) - \pi_i(\sigma_i, \sigma_{-i}) = \sum_k \frac{A_k}{K} \left(\frac{h_i \sigma'_i(s_k)}{H'_k} - \frac{h_i \sigma_i(s_k)}{H_k} \right).$$

For small deviations, the logarithmic potential approximates the payoff difference (up to first order), confirming that the game has a potential and best-response dynamics converge to the unique equilibrium $\sigma^* = (1/K, \dots, 1/K)$. $\square$

B Emission Controller Parameters

Table 3: Complete emission controller parameters from the implementation.

Parameter	Symbol	Value
Maximum supply	$S_{\max}$	21,000,000 QUG
Era-0 total emission	E_0	10,500,000 QUG
Halving period	τ	126,230,400 s
Number of eras	N_{era}	64
Era-0 annual target	A_0	2,625,000 QUG/yr
Seconds per Julian year	Y	31,557,600
Genesis timestamp	t_0	1,771,761,600
Sliding window size	W	1,000 blocks
Max block rate cap	$\lambda_{\max}$	50 bps
Correction smoothing	α	0.8
Correction minimum	$\gamma_{\min}$	0.01
Correction maximum	$\gamma_{\max}$	5.0
Minimum reward	$R_{\min}$	10^{-5} QUG
Maximum reward	$R_{\max}$	2.0 QUG
Development fee	ϕ_{dev}	1.9%
Operator fee	ϕ_{op}	0.1%
Consensus shards	K	4
State shards	K_s	8
VDF iterations	T	2,048
Mining finality depth	d_f	6 blocks
Challenge lifetime	t_{expire}	90 s
PPLNS N -factor	N	2.0
Pool fee	ϕ_{pool}	2.5%

C Notation Summary

Table 4: Summary of notation used throughout the paper.

Symbol	Meaning
$\mathcal{N}$	Set of miners $\{1, \dots, n\}$
$\mathcal{K}$	Set of consensus shards $\{s_1, \dots, s_K\}$
K	Number of consensus shards (default 4)
h_i	Hashrate of miner i (hashes/second)
H	Total network hashrate
H_k	Aggregate hashrate on shard s_k
α	Miner hashrate fraction: h_i/H
σ_i	Miner i 's shard allocation strategy
π_i	Miner i 's expected profit
$R(t)$	Per-block reward at time t
$\lambda(t)$	Measured block rate (blocks/second)
A_k	Annual emission target in era k
E_k	Total emission in era k
$S_{\max}$	Maximum supply (21M QUG)
τ	Halving period (4 years)
$C^*(t)$	Target cumulative emission at time t
$C(t)$	Actual cumulative emission
$\delta(t)$	Emission error: $C(t) - C^*(t)$
$\gamma(t)$	Error correction factor
ϕ_{dev}	Development fee rate (1.9%)
ϕ_{op}	Operator fee rate (0.1%)
d_{w_j}	Difficulty of shares by worker w_j
NE	Nash equilibrium